Atelier 1 – SQL complet

Requêtes, calculs, optimisation et sécurité

Base de données MySQL 8.4 hébergée sur Aiven (connexion SSL)

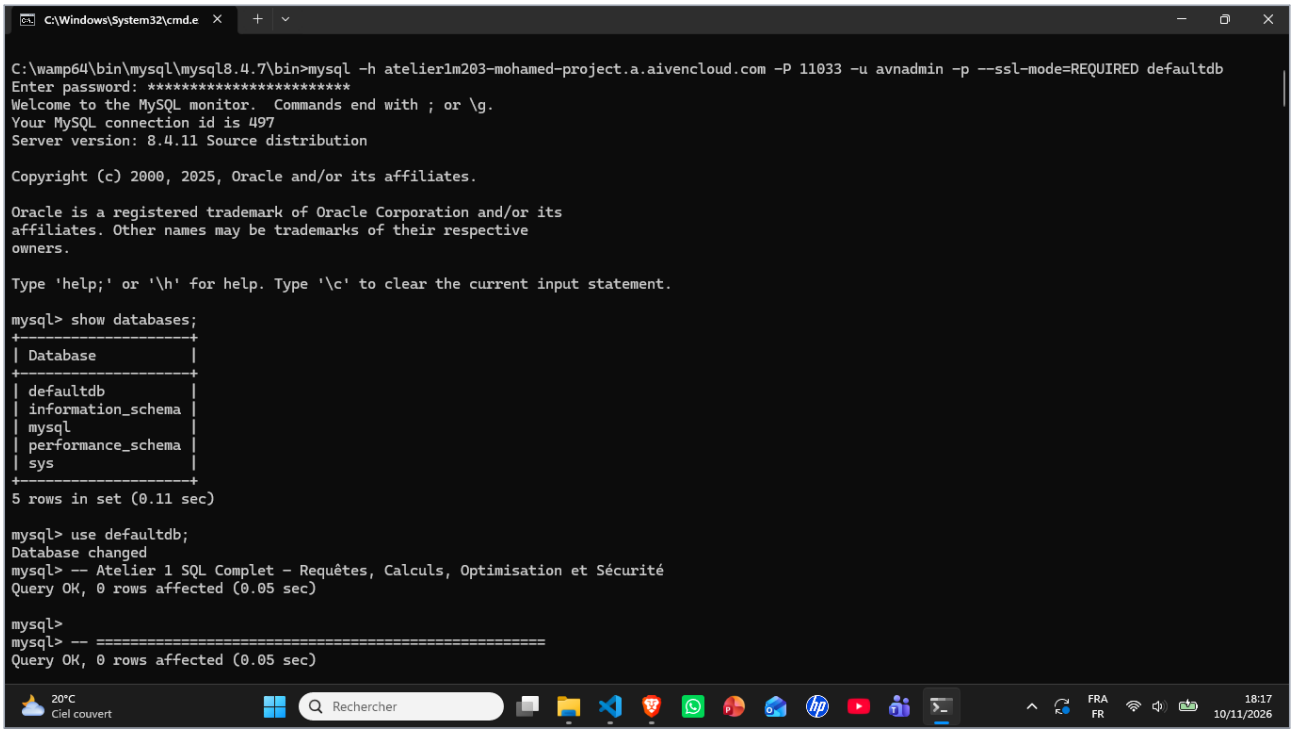
Ce document présente la réalisation de l'atelier : création de la base **atelier_commerce** (clients, produits, commandes, détails des commandes, paiements), puis les jointures, les calculs avec agrégation, les fonctions SQL, les sous-requêtes, l'optimisation (index et vue) et la gestion des droits des utilisateurs. Chaque étape est accompagnée du code SQL exécuté et de la capture d'écran du résultat obtenu dans le terminal.

0. Connexion à la base de données Aiven

La base MySQL a été créée sur **Aiven**. La connexion se fait depuis l'invite de commandes (cmd), ouverte dans le dossier `C:\wamp64\bin\mysql\mysql8.4.7\bin`, avec la commande suivante :

```
mysql -h atelier1m203-mohamed-project.a.aivencloud.com -P 11033 -u avnadmin -p --ssl-mode=REQUIRED defaultdb
```

-h : adresse du serveur (host) ; **-P** : port de la base ; **-u** : utilisateur ; **-p** : demande du mot de passe ; **--ssl-mode=REQUIRED** : connexion chiffrée obligatoire ; **defaultdb** : base ouverte par défaut. Une fois connecté, `show databases;` affiche les bases existantes, puis `use defaultdb;` sélectionne la base de travail. Le script de l'atelier est ensuite collé dans le terminal.



Capture 1 — connexion réussie au serveur MySQL 8.4.11 d'Aiven et début de l'exécution du script.

1. Conception de la base

1.1 – 1.2 Création de la base et des tables. On crée la base **atelier_commerce** puis cinq tables. **clients** et **produits** sont indépendantes ; **commandes** référence un client ; **details_commandes** relie une commande à un produit avec une quantité ; **paiements** référence une commande. Les clés étrangères (FOREIGN KEY) garantissent la cohérence entre les tables.

```
-- 1.1 Création de la base
CREATE DATABASE atelier_commerce;
USE atelier_commerce;

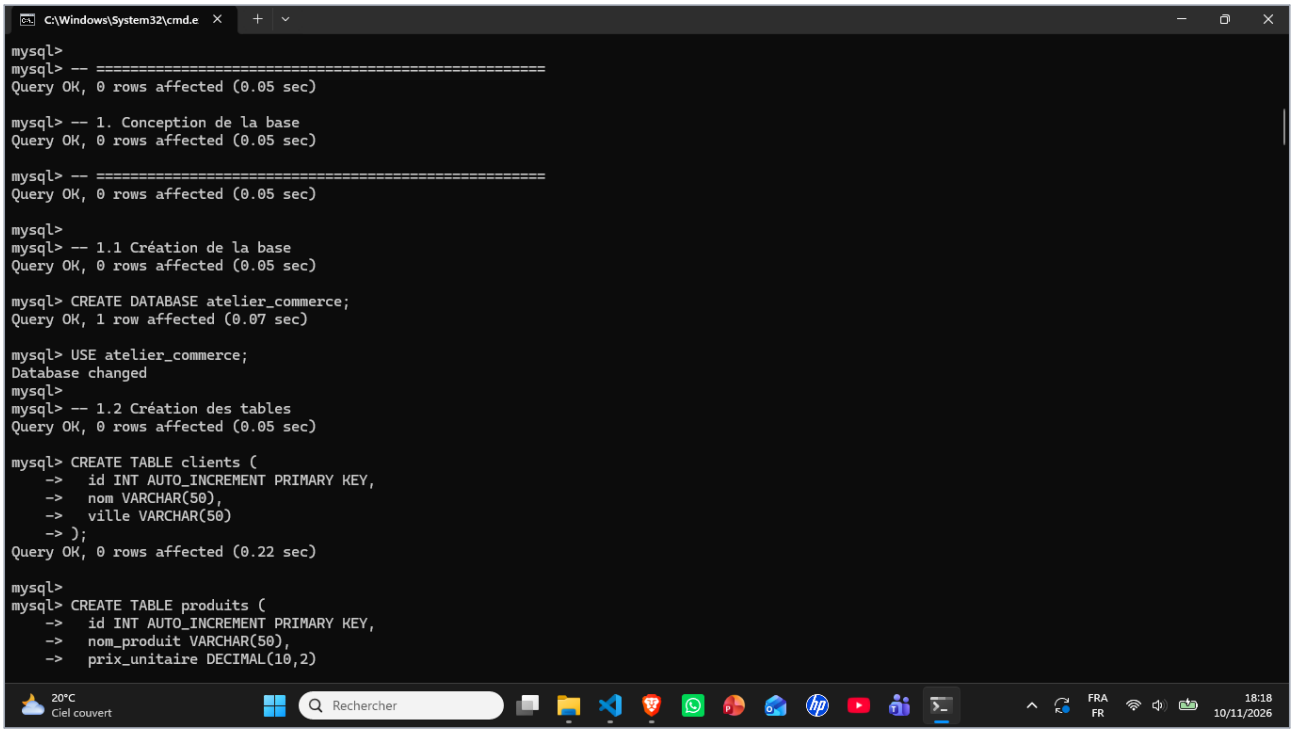
-- 1.2 Création des tables
CREATE TABLE clients (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(50),
  ville VARCHAR(50)
);

CREATE TABLE produits (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom_produit VARCHAR(50),
  prix_unitaire DECIMAL(10,2)
);

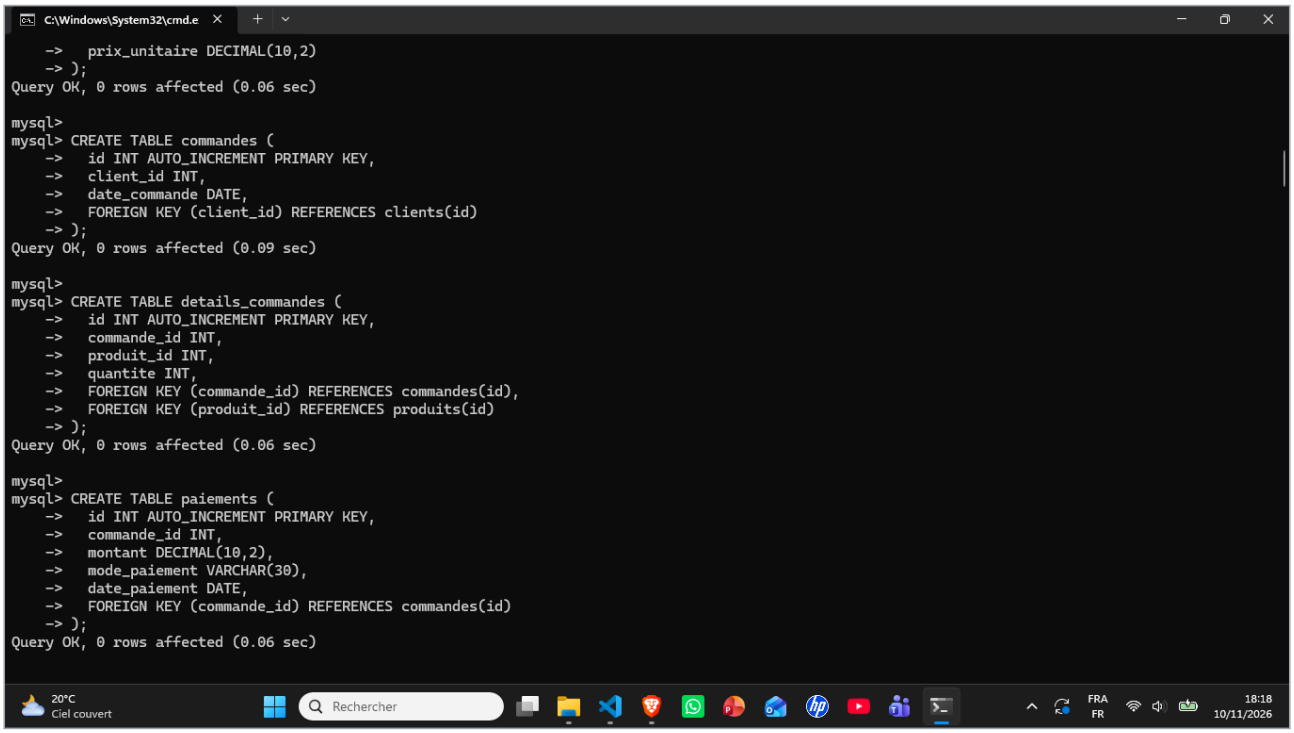
CREATE TABLE commandes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  client_id INT,
  date_commande DATE,
  FOREIGN KEY (client_id) REFERENCES clients(id)
);

CREATE TABLE details_commandes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  commande_id INT,
  produit_id INT,
  quantite INT,
  FOREIGN KEY (commande_id) REFERENCES commandes(id),
  FOREIGN KEY (produit_id) REFERENCES produits(id)
);

CREATE TABLE paiements (
  id INT AUTO_INCREMENT PRIMARY KEY,
  commande_id INT,
  montant DECIMAL(10,2),
  mode_paiement VARCHAR(30),
  date_paiement DATE,
  FOREIGN KEY (commande_id) REFERENCES commandes(id)
);
```



Capture 2 — création de la base atelier_commerce et des tables clients et produits.



Capture 3 — création des tables commandes, details_commandes et paiements (Query OK pour chaque table).

1.3 Insertion des données. On ajoute 4 clients, 4 produits, 4 commandes, 5 lignes de détails et 3 paiements. Le terminal confirme le nombre de lignes insérées (Records) et l'absence d'avertissement (Warnings: 0).

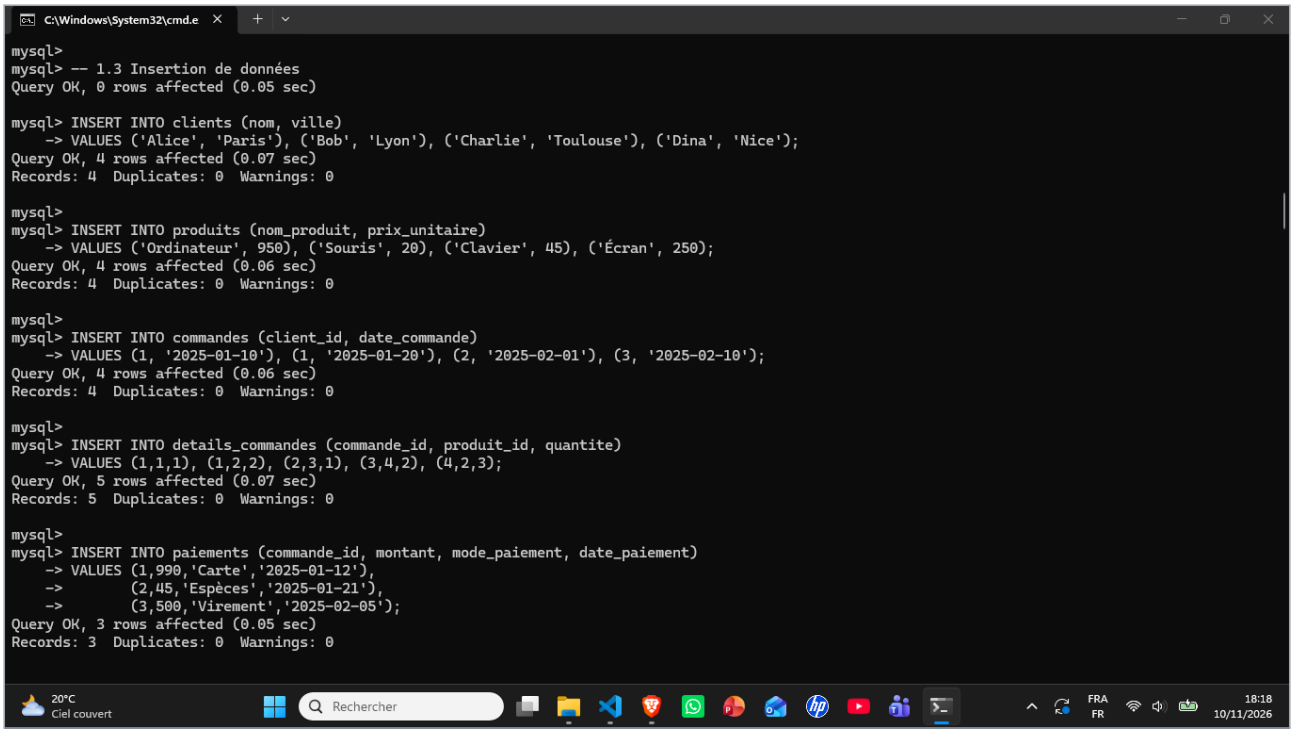
```
-- 1.3 Insertion de données
INSERT INTO clients (nom, ville)
VALUES ('Alice', 'Paris'), ('Bob', 'Lyon'), ('Charlie', 'Toulouse'), ('Dina', 'Nice');

INSERT INTO produits (nom_produit, prix_unitaire)
VALUES ('Ordinateur', 950), ('Souris', 20), ('Clavier', 45), ('Écran', 250);
```

```
INSERT INTO commandes (client_id, date_commande)
VALUES (1, '2025-01-10'), (1, '2025-01-20'), (2, '2025-02-01'), (3, '2025-02-10');

INSERT INTO details_commandes (commande_id, produit_id, quantite)
VALUES (1,1,1), (1,2,2), (2,3,1), (3,4,2), (4,2,3);

INSERT INTO paiements (commande_id, montant, mode_paiement, date_paiement)
VALUES (1,990,'Carte','2025-01-12'),
       (2,45,'Espèces','2025-01-21'),
       (3,500,'Virement','2025-02-05');
```



Capture 4 — insertion des données : 4, 4, 4, 5 et 3 lignes ajoutées, sans avertissement.

2. Requêtes de jointures

Les jointures permettent de combiner plusieurs tables. **INNER JOIN** ne garde que les lignes qui ont une correspondance dans toutes les tables. **LEFT JOIN** garde tous les clients, même sans commande. **RIGHT JOIN** garde toutes les commandes. MySQL n'a pas de FULL JOIN : on le simule avec **LEFT JOIN + UNION + RIGHT JOIN**.

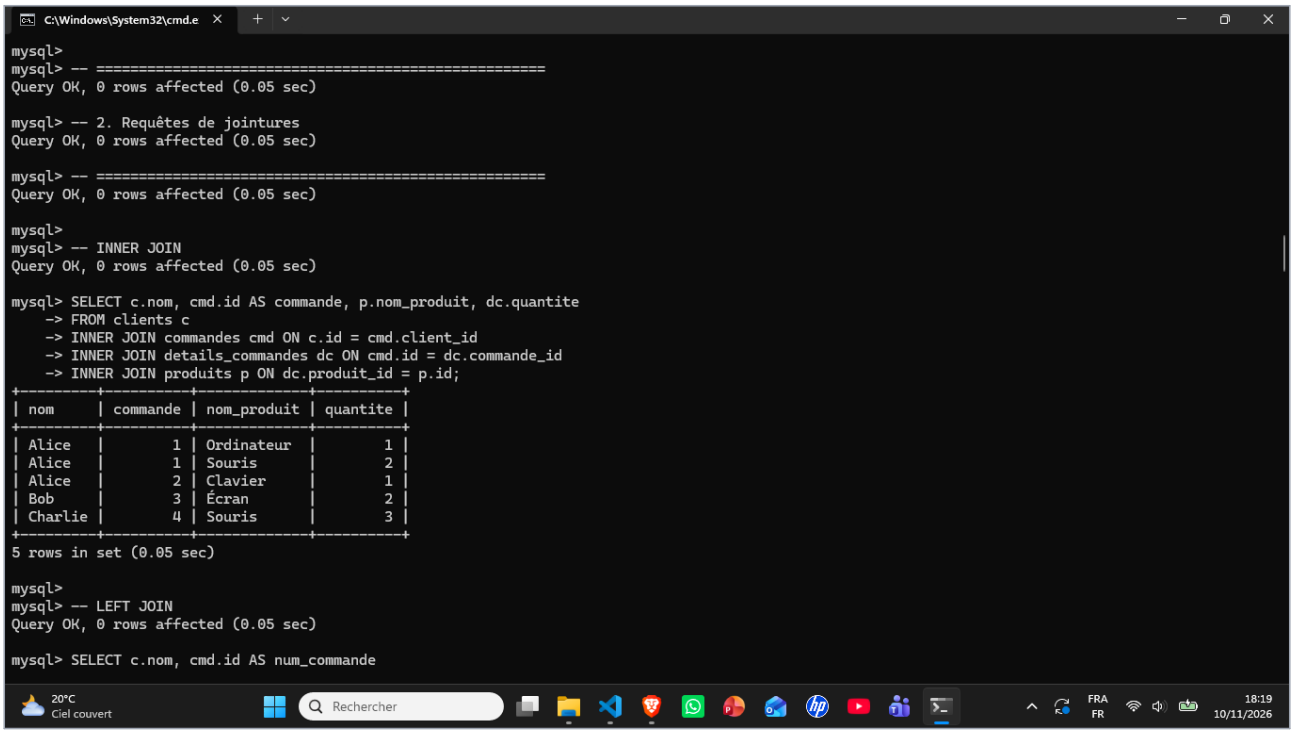
```
-- INNER JOIN
SELECT c.nom, cmd.id AS commande, p.nom_produit, dc.quantite
FROM clients c
INNER JOIN commandes cmd ON c.id = cmd.client_id
INNER JOIN details_commandes dc ON cmd.id = dc.commande_id
INNER JOIN produits p ON dc.produit_id = p.id;

-- LEFT JOIN
SELECT c.nom, cmd.id AS num_commande
FROM clients c
LEFT JOIN commandes cmd ON c.id = cmd.client_id;

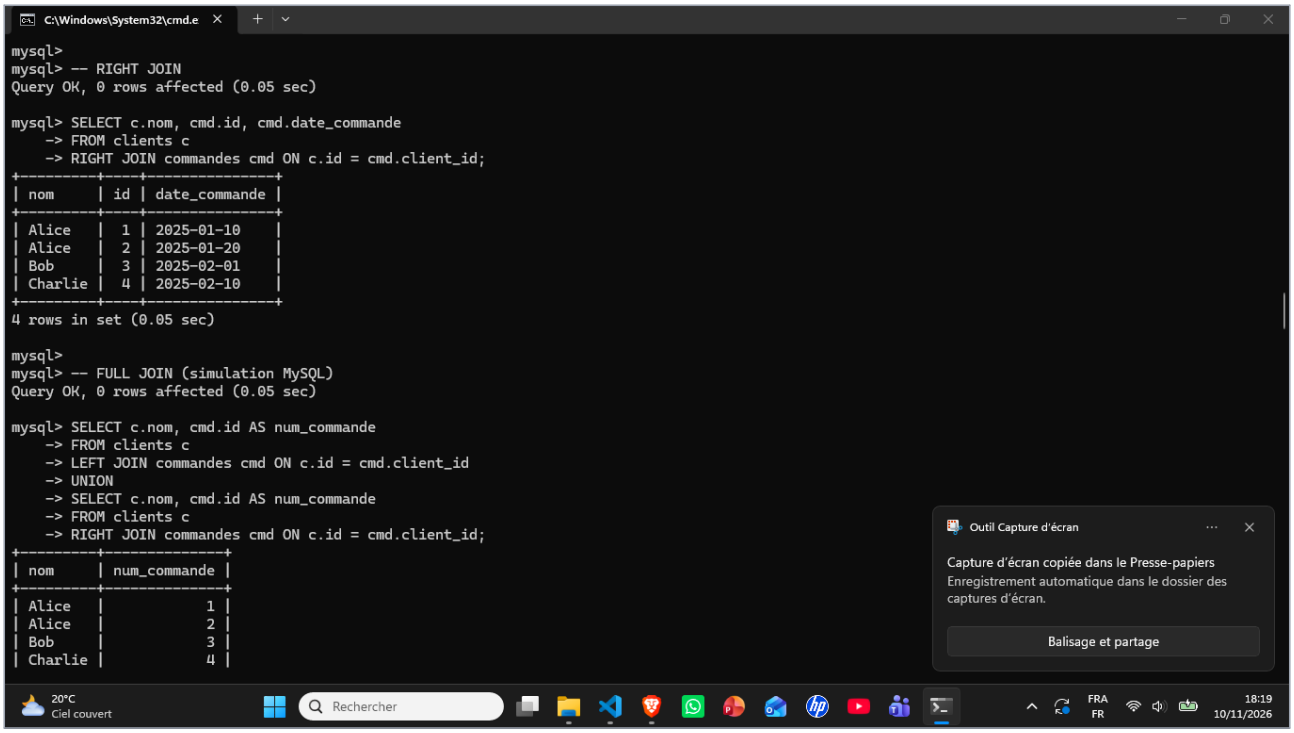
-- RIGHT JOIN
SELECT c.nom, cmd.id, cmd.date_commande
FROM clients c
RIGHT JOIN commandes cmd ON c.id = cmd.client_id;

-- FULL JOIN (simulation MySQL)
SELECT c.nom, cmd.id AS num_commande
FROM clients c
LEFT JOIN commandes cmd ON c.id = cmd.client_id
UNION
SELECT c.nom, cmd.id AS num_commande
FROM clients c
```

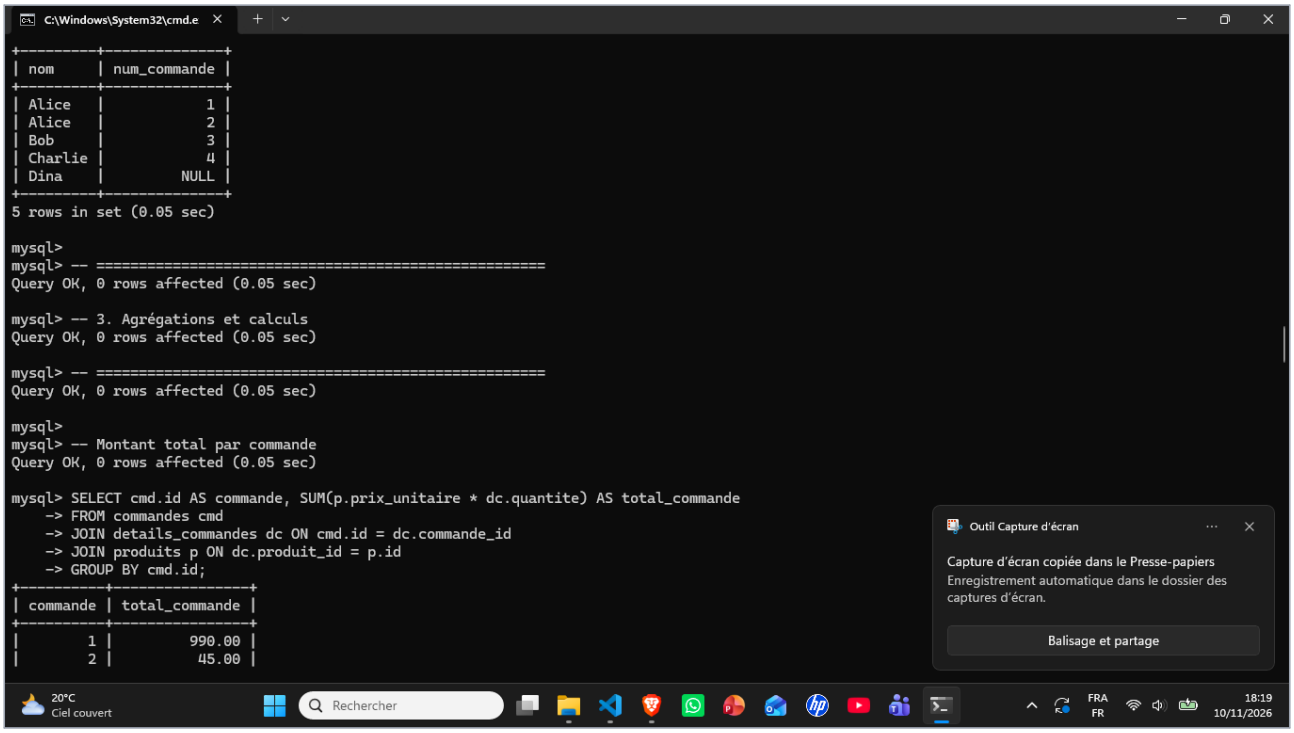
```
RIGHT JOIN commandes cmd ON c.id = cmd.client_id;
```



Capture 5 — INNER JOIN : 5 lignes (client, commande, produit, quantité). Dina n'apparaît pas car elle n'a aucune commande.



Capture 6 — RIGHT JOIN : les 4 commandes avec leur client, puis début de la simulation du FULL JOIN.



Capture 7 — FULL JOIN simulé : 5 lignes. Dina apparaît avec NULL car elle n'a pas de commande.

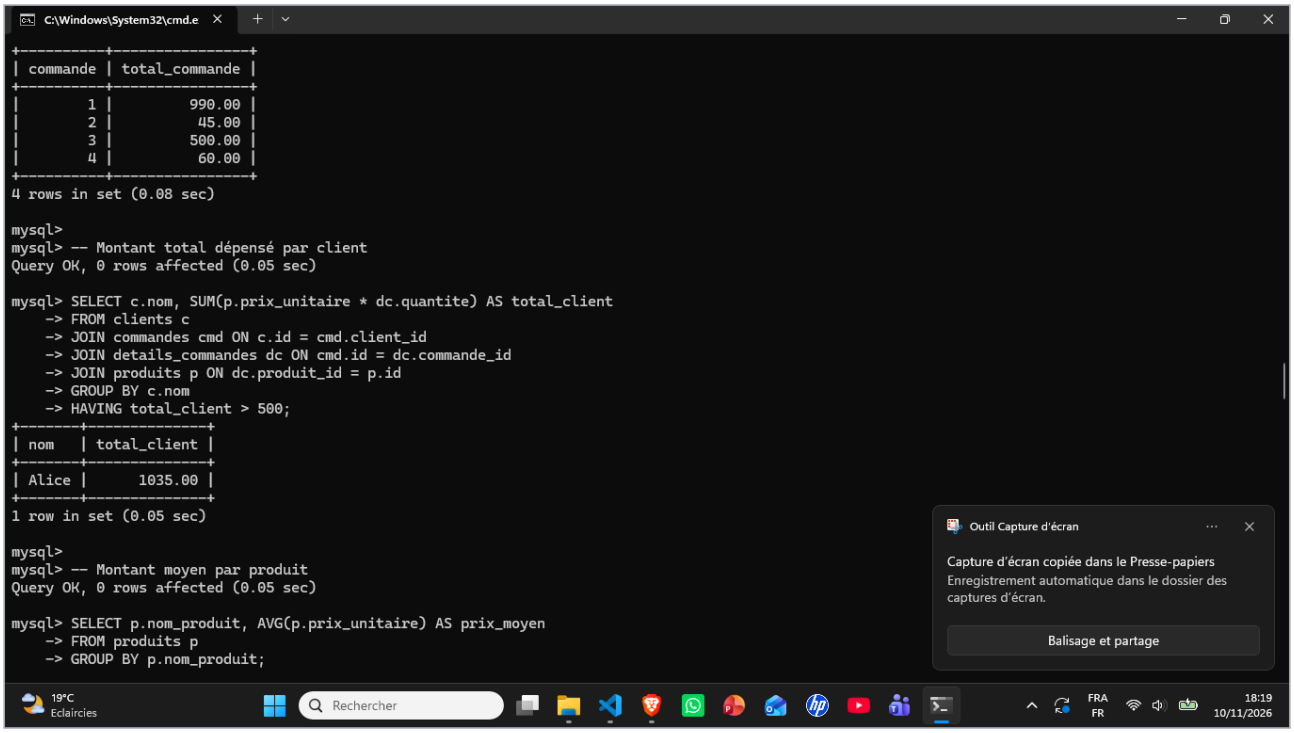
3. Agrégations et calculs

La fonction **SUM** calcule le total (prix × quantité) ; **GROUP BY** regroupe les lignes par commande ou par client ; **HAVING** filtre sur le résultat du calcul ; **AVG** calcule une moyenne. Résultats : commande 1 = 990, commande 2 = 45, commande 3 = 500, commande 4 = 60. Seule Alice dépasse 500 (total 1035). Comme chaque produit n'a qu'un seul prix, le prix moyen par produit est égal à son prix unitaire.

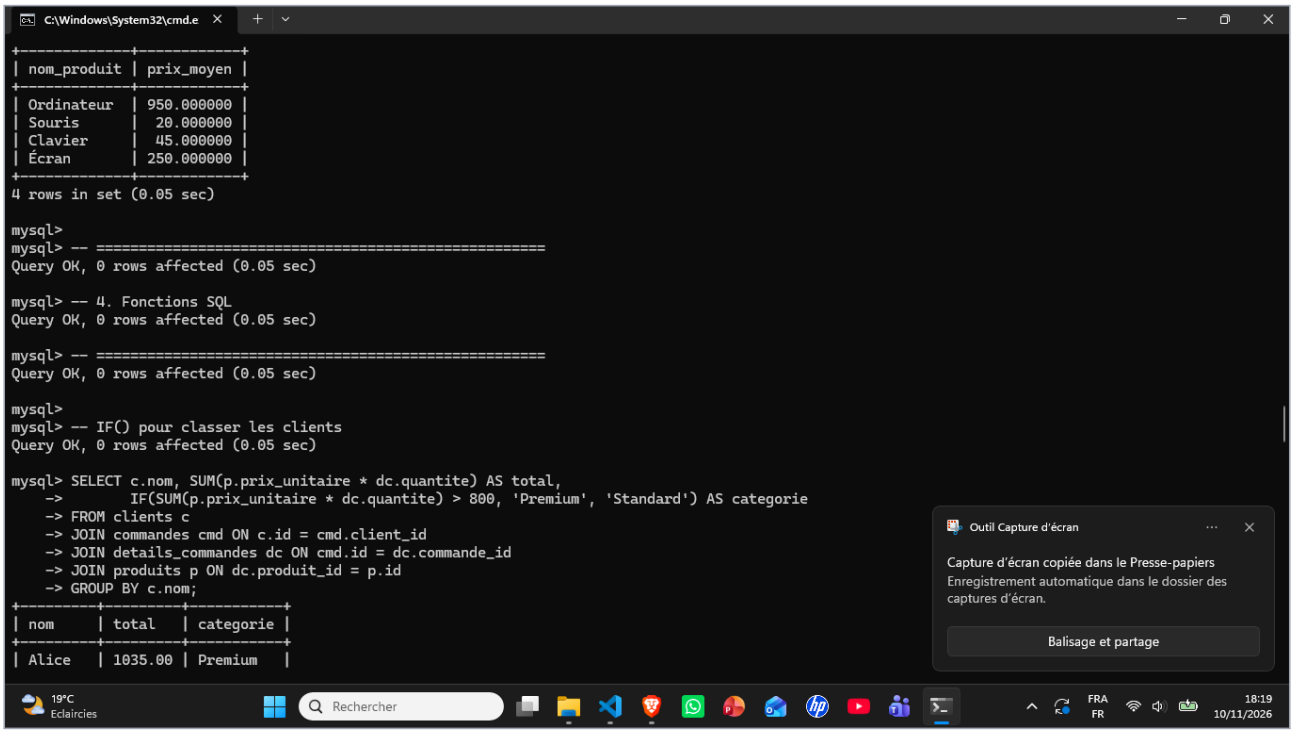
```
-- Montant total par commande
SELECT cmd.id AS commande, SUM(p.prix_unitaire * dc.quantite) AS total_commande
FROM commandes cmd
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY cmd.id;

-- Montant total dépensé par client
SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total_client
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY c.nom
HAVING total_client > 500;

-- Montant moyen par produit
SELECT p.nom_produit, AVG(p.prix_unitaire) AS prix_moyen
FROM produits p
GROUP BY p.nom_produit;
```



Capture 8 — total par commande (990, 45, 500, 60) et clients ayant dépensé plus de 500 : Alice (1035).



Capture 9 — prix moyen par produit, puis classement des clients avec IF().

4. Fonctions SQL

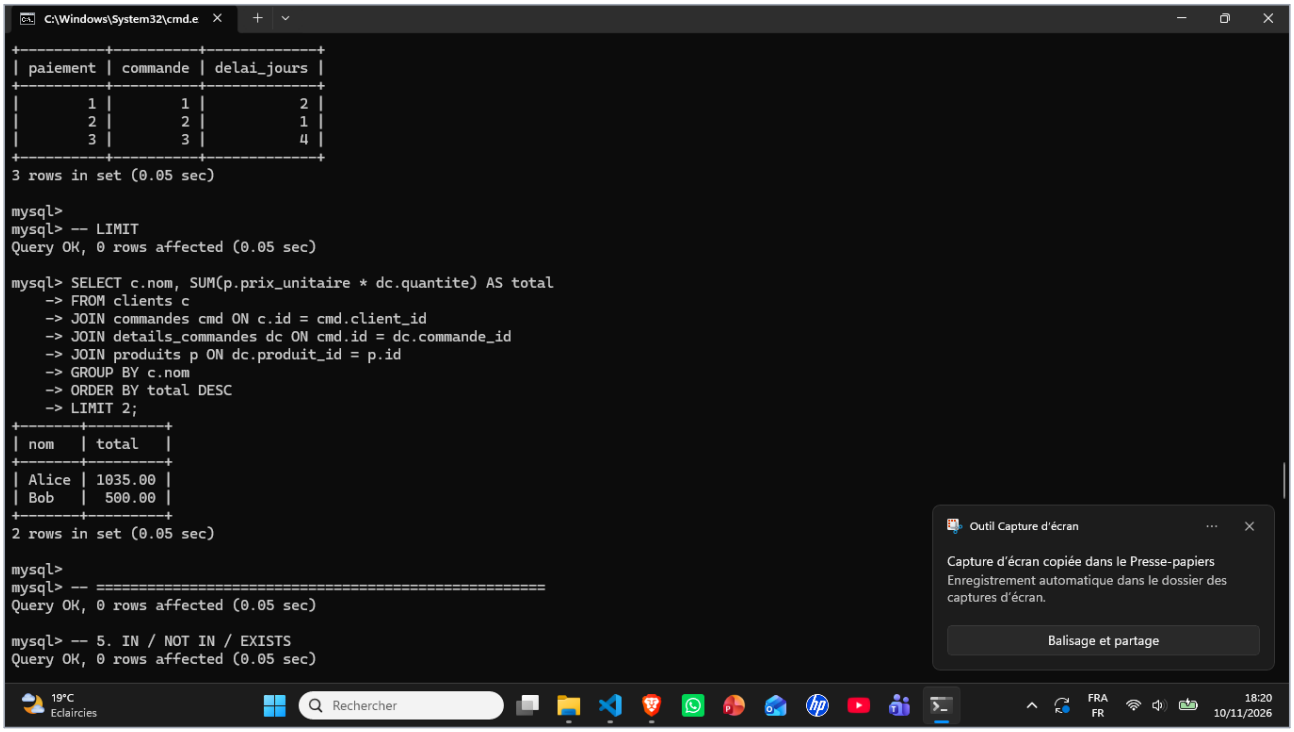
IF() classe un client en « Premium » si son total dépasse 800, sinon « Standard » : Alice (1035) est Premium. **DATEDIFF()** calcule le délai en jours entre la commande et le paiement : 2, 1 et 4 jours. **ORDER BY … DESC LIMIT 2** affiche les deux meilleurs clients : Alice (1035) et Bob (500).

```
-- IF() pour classer les clients
SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total,
       IF(SUM(p.prix_unitaire * dc.quantite) > 800, 'Premium', 'Standard') AS categorie
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
```

```
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY c.nom;

-- Fonctions de date
SELECT p.id AS paiement, cmd.id AS commande,
       DATEDIFF(p.date_paiement, cmd.date_commande) AS delai_jours
FROM paiements p
JOIN commandes cmd ON p.commande_id = cmd.id;

-- LIMIT
SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY c.nom
ORDER BY total DESC
LIMIT 2;
```



Capture 10 — délai entre commande et paiement (2, 1 et 4 jours) et les 2 meilleurs clients avec LIMIT 2.

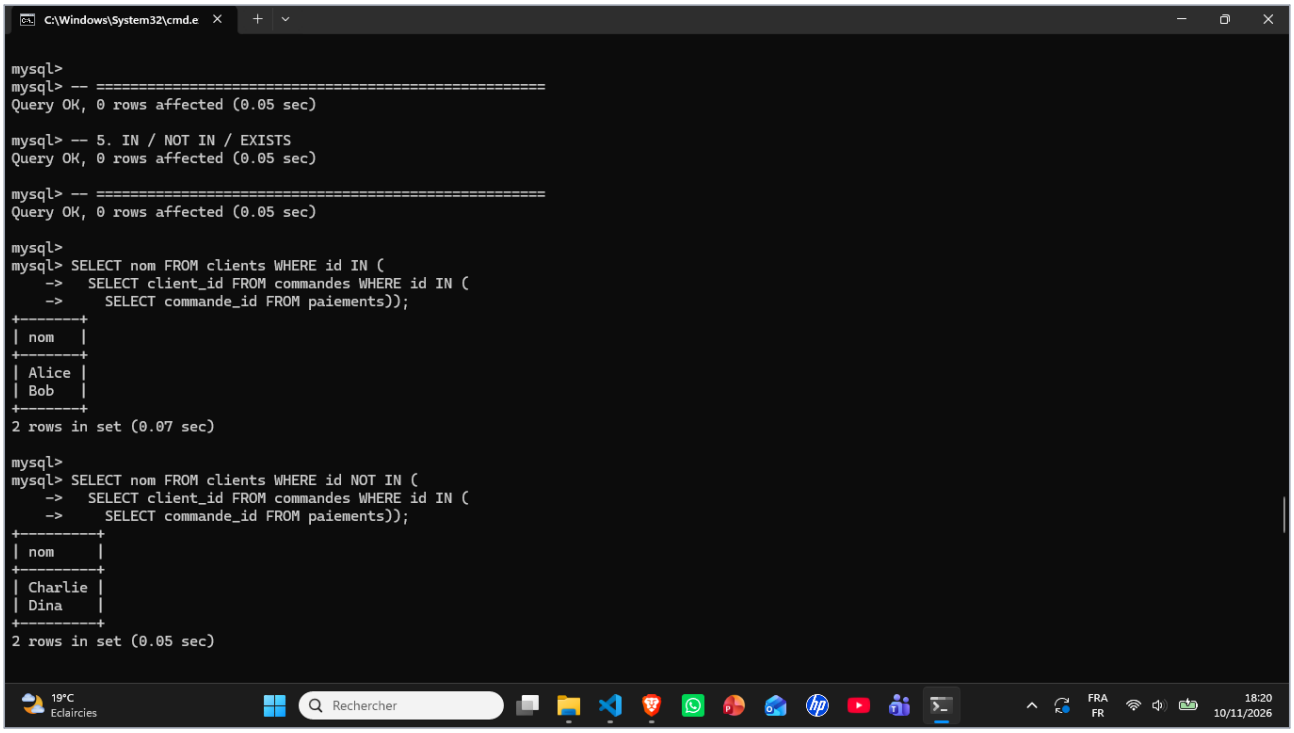
5. IN / NOT IN / EXISTS

Ces trois requêtes utilisent des sous-requêtes. **IN** retourne les clients qui ont au moins une commande payée : Alice et Bob. **NOT IN** retourne le contraire : Charlie (commande non payée) et Dina (aucune commande). **EXISTS** donne le même résultat que IN (Alice, Bob) ; il vérifie simplement qu'une ligne correspondante existe.

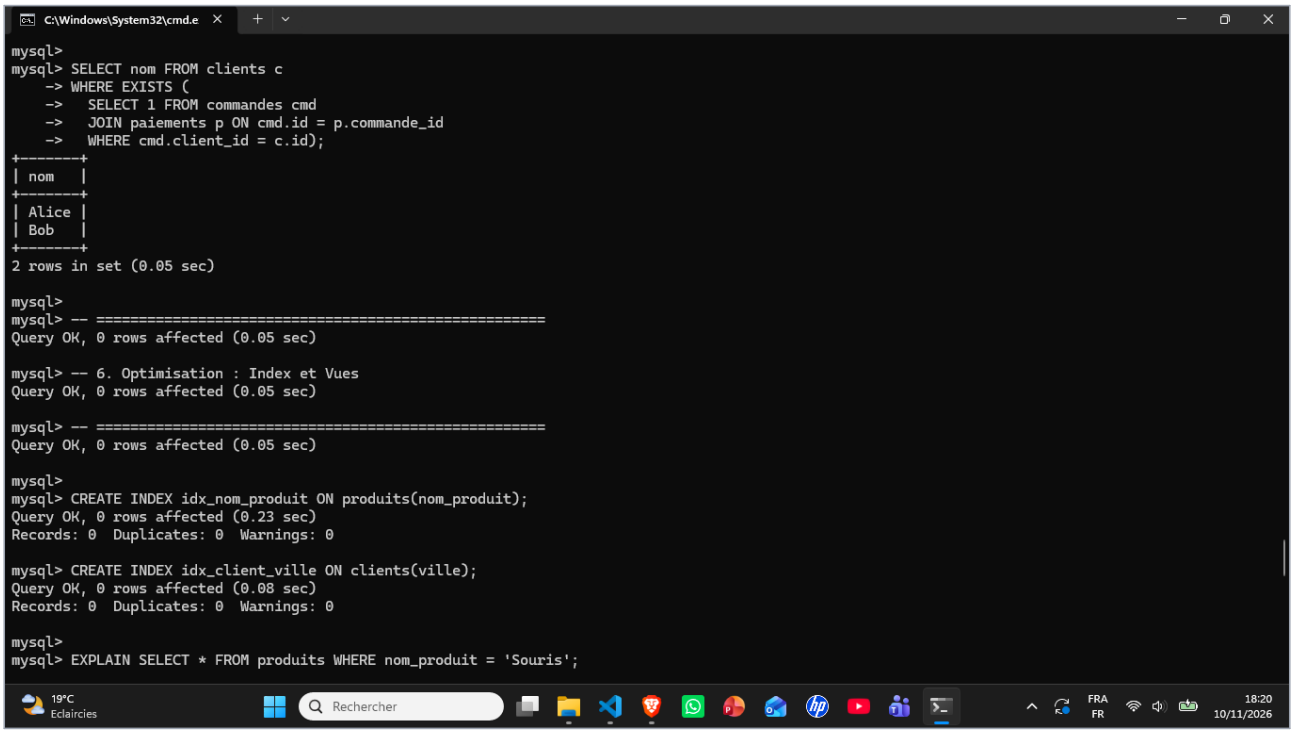
```
SELECT nom FROM clients WHERE id IN (
  SELECT client_id FROM commandes WHERE id IN (
    SELECT commande_id FROM paiements));

SELECT nom FROM clients WHERE id NOT IN (
  SELECT client_id FROM commandes WHERE id IN (
    SELECT commande_id FROM paiements));

SELECT nom FROM clients c
WHERE EXISTS (
  SELECT 1 FROM commandes cmd
  JOIN paiements p ON cmd.id = p.commande_id
  WHERE cmd.client_id = c.id);
```



Capture 11 — IN : Alice et Bob ; NOT IN : Charlie et Dina.



Capture 12 — EXISTS : Alice et Bob ; création des deux index et requête EXPLAIN.

6. Optimisation : index et vues

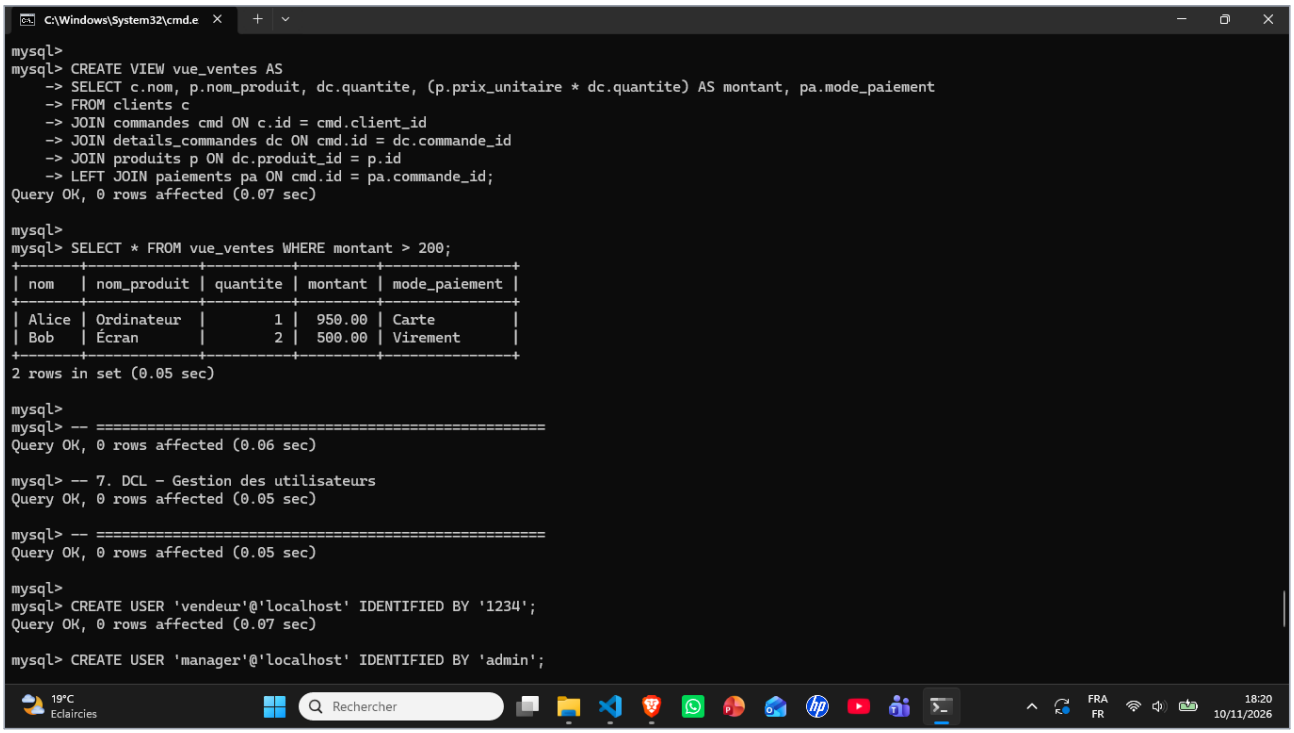
Un **index** accélère les recherches sur une colonne (ici nom_produit et ville). **EXPLAIN** montre comment MySQL exécute la requête et si l'index est utilisé. Une **vue** est une requête enregistrée qui se comporte comme une table : **vue_ventes** regroupe client, produit, quantité, montant et mode de paiement. Avec « montant > 200 », elle retourne Alice (Ordinateur, 950, Carte) et Bob (Écran, 500, Virement).

```
CREATE INDEX idx_nom_produit ON produits(nom_produit);
CREATE INDEX idx_client_ville ON clients(ville);
```

```
EXPLAIN SELECT * FROM produits WHERE nom_produit = 'Souris';

CREATE VIEW vue_ventes AS
SELECT c.nom, p.nom_produit, dc.quantite, (p.prix_unitaire * dc.quantite) AS montant, pa.mode_paiement
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
LEFT JOIN paiements pa ON cmd.id = pa.commande_id;

SELECT * FROM vue_ventes WHERE montant > 200;
```



Capture 13 — création de la vue vue_ventes et son interrogation (montant > 200), puis création de l'utilisateur vendeur.

7. DCL – Gestion des utilisateurs

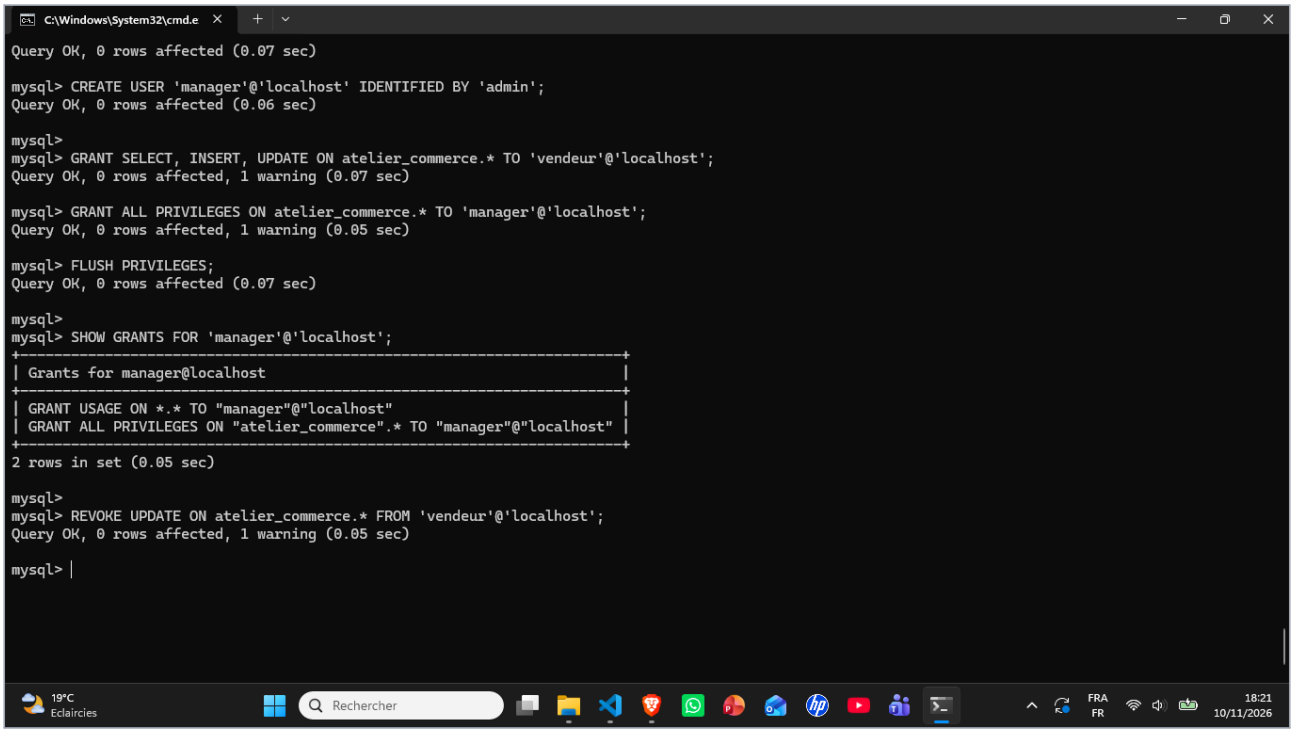
Le DCL gère les droits d'accès. On crée deux utilisateurs : **vendeur** (droits SELECT, INSERT, UPDATE) et **manager** (tous les droits sur la base atelier_commerce). **FLUSH PRIVILEGES** recharge les droits. **SHOW GRANTS** confirme que manager a bien ALL PRIVILEGES. Enfin, **REVOKE** retire le droit UPDATE à vendeur, qui ne garde alors que SELECT et INSERT.

```
CREATE USER 'vendeur'@'localhost' IDENTIFIED BY '1234';
CREATE USER 'manager'@'localhost' IDENTIFIED BY 'admin';

GRANT SELECT, INSERT, UPDATE ON atelier_commerce.* TO 'vendeur'@'localhost';
GRANT ALL PRIVILEGES ON atelier_commerce.* TO 'manager'@'localhost';
FLUSH PRIVILEGES;

SHOW GRANTS FOR 'manager'@'localhost';

REVOKE UPDATE ON atelier_commerce.* FROM 'vendeur'@'localhost';
```



Capture 14 — création de manager, attribution des droits, SHOW GRANTS et retrait du droit UPDATE à vendeur.

Conclusion

Toutes les requêtes de l'atelier ont été exécutées avec succès sur la base MySQL hébergée sur Aiven : création des tables, insertion des données, jointures, agrégations, fonctions, sous-requêtes, index, vue et gestion des droits.